

COMPUTER SCIENCE TRIPOS Part IB – 2024 – Paper 4

4 Prolog (ijl20)

In your answers ensure each relation which you define has a comment giving a declarative reading of its behaviour. Avoid unnecessary use of *cut* or other extra-logical relations. The library relations `\=`, `is`, and `atomic(X)` may be used, the last succeeding if `X` is a number or atom e.g. `42` or `abc`. Other relations should not be assumed.

Answer: Note: use of `cut` will be -1 mark for each part where it is used.

rules, arithmetic

- (a) Define a relation `eval/2` to reduce arithmetic terms, so that e.g. `eval(1+2*3,N)` succeeds with `N=7`. Atoms should reduce to themselves, so e.g. `eval(a,Ans)` succeeds with `Ans=a`. [4 marks]

Answer:

```
% eval(+X,?X) succeeds if X is atomic i.e. an atom or number.
eval(X,X) :- atomic(X).
```

```
% each of these eval(+arith_exp, ?result) succeeds if the first argument is
% a valid arithmetic expression and A the value of the reduced expression.
```

```
eval(X+Y,A) :- eval(X,X1), eval(Y,Y1), A is X1+Y1.
```

```
eval(X-Y,A) :- eval(X,X1), eval(Y,Y1), A is X1-Y1.
```

```
eval(X*Y,A) :- eval(X,X1), eval(Y,Y1), A is X1*Y1.
```

```
eval(X/Y,A) :- eval(X,X1), eval(Y,Y1), A is X1/Y1.
```

Declarative comments for `atomic` and arithmetic cases (2 marks). `atomic` clause (1 mark). Arithmetic clauses (1 mark). Recursive `eval` is not required in this part, but will be needed later. `eval(X,A) :- A is X.` is not an acceptable answer due to the error in handling atoms.

rules, function
term reduction

- (b) Extend `eval` to allow function calls within arithmetic terms, such that `2 * apply(inc, [2+3])` reduces to `12`. We will declare functions as Prolog facts in a `fun/2` relation, e.g. `inc` above will be specified by the fact `fun(apply(inc,[N]), N+1)`. Note the function arguments are held in a list, `[N]` in this example, to support multi-argument functions. [6 marks]

Answer:

```
% eval(apply(+Fn,Args),?Ans) succeeds if apply(Fn,Args) unifies with
% the first argument of a fun relation and Ans is the reduced value
% of that functional term.
```

```
eval(apply(Fn,Args),Ans) :-
```

```
    eval(Args, Args2), fun(apply(Fn,Args2), Body), eval(Body, Ans).
```

Correct call to the `fun/2` relation (1 mark). Working `eval` rule for function reduction without the `inc` functor hard-coded into the `eval` rule (2 marks), succinct solution (1 mark). Complete declarative comments (2 marks).

rules, constant
terms

- (c) Extend relation `eval` to support a `==/2` operator, which reduces term `A==B` to `true` if `A` and `B` reduce to the same number or atom and `false` otherwise. For example `eval(1+3==2+2,Ans)` succeeds with `Ans=true`. [3 marks]

Answer:

```
% eval(X==Y,Ans) succeeds with Ans = 'true' if X unifies with Y, otherwise
% succeeds with Ans = 'false'.
eval(X==Y,true) :- eval(X,X1), eval(Y,Y1), X1=Y1.
eval(X==Y,false) :- eval(X,X1), eval(Y,Y1), X1\=Y1.
```

Declarative comment (1 mark). Working eval clauses for X==Y (1 mark). Succinct eval definition for == (1 mark).

rules, functional
reduction

- (d) Extend relation eval to support terms of the form if(Condition,Then,Else). These if/3 terms should reduce to the reduction of either the Then term or the Else term determined by Condition reducing to true or false. For example eval(if(1+2==4+5,a,b),Ans) succeeds with Ans=b. Add a fact to the fun relation specifying the *factorial* function such that eval(apply(fact,[5]),N) succeeds with N=120. [7 marks]

Answer:

```
% eval(+if(Cond,Then,Else), Ans) succeeds with Ans equal the reduction of
% the Then part if Cond reduces to 'true', or the reduction of the Else
% part if Cond reduces to 'false'.
eval(if(Cond,Then,_Else),A) :- eval(Cond,true), eval(Then,A).
eval(if(Cond,_Then,Else),A) :- eval(Cond,false), eval(Else,A).
```

```
% This fact defines the head and body of the factorial function.
fun(apply(fact,[N]), if(N==1,1,N * apply(fact,[N-1]))).
```

Declarative comment for eval(if(Cond,Then,Else),Ans) (1 mark). In comment include success for both true and false (1 mark). Working reduction of if-then-else to 'true'/'false' (1 mark). Succinct 2 rules (1 mark). Declarative comment for fun(fact(N),Body) fact (1 mark). Working term for fact(N) body (2 marks).
